

Department–Employee One-to-Many Mapping Spring Boot Project

1. Project Title

Department Employee Management System using One-to-Many Mapping in Spring Boot

2. Purpose of the Project

The purpose of this project is to help mentees implement a clean Spring Boot REST API that demonstrates a **one-to-many relationship** between `Department` and `Employee`.

The system must allow authenticated users to manage departments and their employees using REST APIs.

3. Core Relationship

The project must implement the following relationship:

One `Department` can have many `Employees`.
One `Employee` belongs to one `Department`.

Example:

```
Engineering Department
├─ Rahul Sharma
├─ Priya Mehta
└─ Sneha Patil
```

4. Technology Requirements

The project must use:

Area	Technology
Backend Framework	Spring Boot
Language	Java

Area	Technology
Database	MySQL
ORM	Spring Data JPA / Hibernate
API Type	REST API
Security	Spring Security with Basic Authentication
API Documentation	Swagger / OpenAPI
Validation	Jakarta Validation
Boilerplate Reduction	Lombok
Mapping	ModelMapper
Logging	SLF4J Logger
Build Tool	Maven

5. Functional Scope

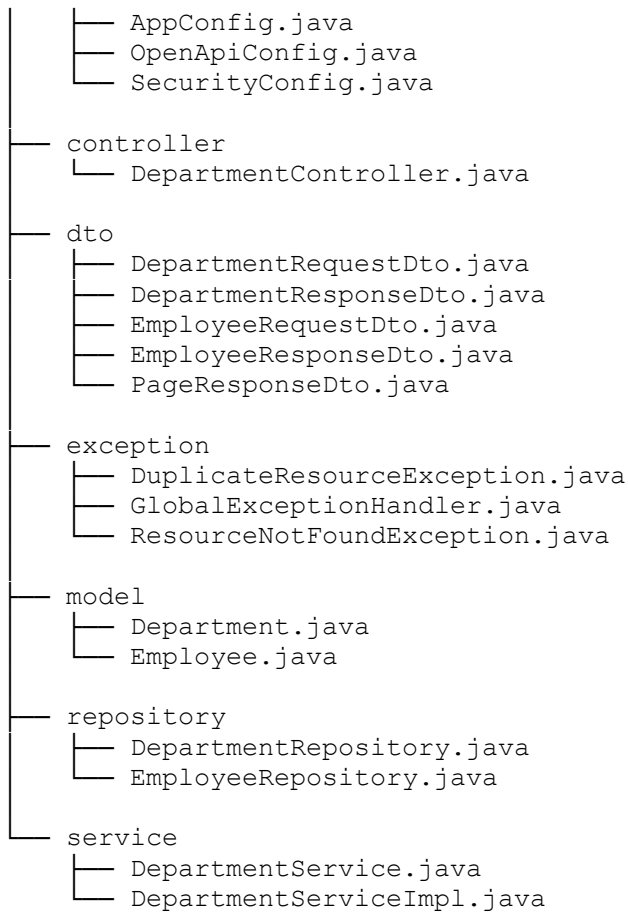
The system must support:

1. Create a department with multiple employees.
 2. View all departments.
 3. View department by ID.
 4. View departments with pagination.
 5. Update department details and employee list.
 6. Delete a department.
 7. Delete employees automatically when their department is deleted.
 8. Prevent duplicate department names.
 9. Prevent duplicate employee emails.
 10. Validate request data.
 11. Handle exceptions globally.
 12. Secure APIs using role-based access.
 13. Document and test APIs using Swagger.
 14. Maintain logs for major operations and errors.
-

6. Required Project Structure

Mentees must create the project using the following package structure:

```
src/main/java/com/swabhav/demo
├── DemoApplication.java
└── config
```



7. Database Requirements

7.1 Database Name

The database must be named:

`one_to_many_demo`

7.2 Tables

The application must generate the following tables:

`departments`
`employees`

7.3 Department Table Requirements

The `departments` table must contain:

Column	Purpose
id	Unique department ID
department_name	Department name
location	Department location

7.4 Employee Table Requirements

The `employees` table must contain:

Column	Purpose
id	Unique employee ID
employee_name	Employee name
email	Employee email
salary	Employee salary
department_id	Foreign key linked to department

8. Entity Requirements

8.1 Department Entity

Class name:

`Department`

Package:

`com.swabhav.demo.model`

Purpose:

Represents a department record in the database.

Required fields:

Field Name	Data Type	Requirement
id	Long	Primary key
departmentName	String	Required and unique
location	String	Required

Field Name	Data Type	Requirement
employees	List<Employee>	One-to-many relationship

Relationship requirement:

Department must have a one-to-many relationship with Employee.

Business rule:

When a department is deleted, all employees under that department must also be deleted.

8.2 Employee Entity

Class name:

Employee

Package:

com.swabhav.demo.model

Purpose:

Represents an employee record in the database.

Required fields:

Field Name	Data Type	Requirement
id	Long	Primary key
employeeName	String	Required
email	String	Required and unique
salary	Double	Required and greater than zero
department	Department	Many-to-one relationship

Relationship requirement:

Employee must belong to one Department.

9. DTO Requirements

Entities must not be directly exposed through the controller layer.
The system must use request and response DTOs.

9.1 DepartmentRequestDto

Purpose:

Used when creating or updating a department.

Required fields:

Field Name	Data Type	Validation Requirement
departmentName	String	Required
location	String	Required
employees	List<EmployeeRequestDto>	Required, must contain at least one employee

JSON field requirement:

departmentName must appear as department_name in request JSON.

9.2 DepartmentResponseDto

Purpose:

Used while sending department data in API responses.

Required fields:

Field Name	Data Type
id	Long
departmentName	String
location	String
employees	List<EmployeeResponseDto>

JSON field requirement:

departmentName must appear as department_name in response JSON.

9.3 EmployeeRequestDto

Purpose:

Used when creating or updating employee data inside a department request.

Required fields:

Field Name	Data Type	Validation Requirement
employeeName	String	Required
email	String	Required and valid email format
salary	Double	Required and greater than zero

JSON field requirement:

employeeName must appear as employee_name in request JSON.

9.4 EmployeeResponseDto

Purpose:

Used while sending employee data in API responses.

Required fields:

Field Name	Data Type
id	Long
employeeName	String
email	String
salary	Double

JSON field requirement:

employeeName must appear as employee_name in response JSON.

9.5 PageResponseDto

Purpose:

Used for paginated API responses.

Required generic type:

PageResponseDto<T>

Required fields:

Field Name	Data Type
content	List<T>
pageNumber	int
pageSize	int
totalElements	long
totalPages	int
lastPage	boolean

10. Repository Requirements

10.1 DepartmentRepository

Package:

`com.swabhav.demo.repository`

Purpose:

Handles database operations for Department.

Must support:

Basic CRUD operations

Pagination

Duplicate department name checking

Duplicate department name checking while updating

Required method names:

`existsByDepartmentName`

`existsByDepartmentNameAndIdNot`

10.2 EmployeeRepository

Package:

`com.swabhav.demo.repository`

Purpose:

Handles database operations for Employee.

Must support:

Basic CRUD operations
Duplicate employee email checking
Duplicate employee email checking while updating

Required method names:

existsByEmail
existsByEmailAndIdNot

11. Service Layer Requirements

11.1 DepartmentService

Package:

com.swabhav.demo.service

Type:

Interface

Purpose:

Defines department-related business operations.

Required method names:

Method Name	Purpose
createDepartment	Create department with employees
getAllDepartments	Fetch all departments
getAllDepartmentsWithPagination	Fetch departments with pagination
getDepartmentById	Fetch department by ID
updateDepartment	Update department and employees
deleteDepartment	Delete department by ID

11.2 DepartmentServiceImpl

Package:

`com.swabhav.demo.service`

Type:

Service implementation class

Purpose:

Contains business logic for department and employee management.

Required dependencies:

Dependency Name	Purpose
DepartmentRepository	Department database operations
EmployeeRepository	Employee database operations
ModelMapper	DTO and entity mapping

Required public methods:

```
createDepartment
getAllDepartments
getAllDepartmentsWithPagination
getDepartmentById
updateDepartment
deleteDepartment
```

Required private helper methods:

```
findDepartmentById
attachEmployeesToDepartment
validateEmployeeEmailsForCreate
validateEmployeeEmailsForUpdate
validatePagination
```

12. Business Rules

12.1 Create Department

While creating a department:

1. Department name must be unique.
 2. Employee emails must be unique.
 3. At least one employee must be provided.
 4. Each employee must be linked to the department.
 5. Department and employees must be saved together.
 6. Response must return department details with employee details.
-

12.2 Get All Departments

The system must return all departments with their employee details.

12.3 Get Department By ID

The system must:

1. Accept department ID.
 2. Search department by ID.
 3. Return department details with employees if found.
 4. Return a proper error response if not found.
-

12.4 Pagination

The system must support paginated department listing.

Pagination request parameters:

Parameter	Type	Default
pageNumber	int	0
pageSize	int	5

Pagination validation rules:

Rule	Error Condition
Page number must not be negative	pageNumber < 0
Page size must be greater than zero	pageSize <= 0
Page size must not exceed 100	pageSize > 100

12.5 Update Department

While updating a department:

1. Department must exist.
 2. New department name must not belong to another department.
 3. Employee emails must not duplicate existing employees outside the current department.
 4. Department name must be updated.
 5. Department location must be updated.
 6. Employee list must be updated.
 7. Old employees not present in the new list must be removed.
 8. New employees must be linked to the same department.
 9. Response must return updated department details.
-

12.6 Delete Department

While deleting a department:

1. Department must exist.
 2. Department must be deleted by ID.
 3. Employees belonging to that department must also be deleted.
 4. Response must return no body.
-

13. Controller Requirements

13.1 DepartmentController

Package:

`com.swabhav.demo.controller`

Purpose:

Exposes REST APIs for department management.

Base URL:

`/api/departments`

Required dependency:

DepartmentService

Required API methods:

API Method Name	HTTP Method	Endpoint	Access
createDepartment	POST	/api/departments	ADMIN
getAllDepartments	GET	/api/departments	USER, ADMIN
getAllDepartmentsWithPagination	GET	/api/departments/page	USER, ADMIN
getDepartmentById	GET	/api/departments/{id}	USER, ADMIN
updateDepartment	PUT	/api/departments/{id}	ADMIN
deleteDepartment	DELETE	/api/departments/{id}	ADMIN

14. API Requirements

14.1 Create Department

HTTP method:

POST

Endpoint:

/api/departments

Access:

ADMIN only

Request body must include:

department_name
location
employees

Each employee must include:

employee_name
email
salary

Expected response status:

200 OK or 201 CREATED

Preferred response status:

201 CREATED

14.2 Get All Departments

HTTP method:

GET

Endpoint:

/api/departments

Access:

USER or ADMIN

Response:

List of departments with employees

14.3 Get Departments With Pagination

HTTP method:

GET

Endpoint:

/api/departments/page

Access:

USER or ADMIN

Request parameters:

pageNumber
pageSize

Response:

Paginated department response

14.4 Get Department By ID

HTTP method:

GET

Endpoint:

/api/departments/{id}

Access:

USER or ADMIN

Path variable:

id

Response:

Single department with employees

14.5 Update Department

HTTP method:

PUT

Endpoint:

/api/departments/{id}

Access:

ADMIN only

Path variable:

id

Request body must include:

```
department_name
location
employees
```

Expected response:

Updated department with employees

14.6 Delete Department

HTTP method:

DELETE

Endpoint:

```
/api/departments/{id}
```

Access:

ADMIN only

Expected response status:

204 NO CONTENT

15. Validation Requirements

15.1 Department Validation

Field	Rule
departmentName	Required
location	Required
employees	Required and must contain at least one employee

15.2 Employee Validation

Field	Rule
employeeName	Required
email	Required and must be valid email format
salary	Required and greater than zero

16. Exception Handling Requirements

16.1 Required Custom Exceptions

The project must include:

```
ResourceNotFoundException  
DuplicateResourceException
```

16.2 ResourceNotFoundException

Purpose:

Used when a requested department is not found.

Expected status:

```
404 NOT FOUND
```

16.3 DuplicateResourceException

Purpose:

Used when:

```
Department name already exists.  
Employee email already exists.
```

Expected status:

```
409 CONFLICT
```

16.4 GlobalExceptionHandler

Purpose:

Handles all application exceptions globally.

Required handler method names:

Method Name	Handles
handleResourceNotFound	Missing department
handleDuplicateResource	Duplicate department or employee
handleValidation	DTO validation errors
handleTypeMismatch	Invalid path variables or request parameters
handleInvalidJson	Invalid request body
handleDataIntegrity	Database constraint violations
handleAccessDenied	Unauthorized role access
handleGeneric	Unexpected errors

16.5 Standard Error Response Format

Every normal error response must include:

```
timestamp
status
error
message
```

Validation error response must include:

```
timestamp
status
error
messages
```

17. Security Requirements

17.1 Authentication Type

The project must use:

HTTP Basic Authentication

JWT must not be used.

17.2 Required Users

Username	Password	Role
admin	admin123	ADMIN
user	user123	USER

17.3 Role-Based Access

HTTP Method	Endpoint Pattern	Access
GET	/api/departments/**	USER or ADMIN
POST	/api/departments/**	ADMIN only
PUT	/api/departments/**	ADMIN only
DELETE	/api/departments/**	ADMIN only

17.4 Public Endpoints

The following endpoints must be accessible without login:

/swagger-ui.html
/swagger-ui/**
/v3/api-docs/**

18. Swagger Requirements

Swagger must be configured for the project.

Swagger UI URL:

`http://localhost:8080/swagger-ui.html`

OpenAPI JSON URL:

`http://localhost:8080/v3/api-docs`

Swagger must show all department APIs.

Swagger must support Basic Authentication.

Swagger documentation title:

`Department Employee One-to-Many API`

Swagger version:

`1.0`

19. Logging Requirements

The project must include logger statements in:

`DepartmentController`
`DepartmentServiceImpl`
`GlobalExceptionHandler`

Logging must be added for:

Area	Required Logging
API request received	Controller methods
Create operation	Service layer
Read operation	Service layer
Update operation	Service layer
Delete operation	Service layer
Duplicate data	Service layer
Resource not found	Service layer or exception handler

Area	Required Logging
Validation failure	Exception handler
Access denied	Exception handler
Unexpected error	Exception handler

No separate logging module is required.

20. Configuration Requirements

20.1 AppConfig

Purpose:

Configure application-level beans.

Required bean:

Mapper

20.2 SecurityConfig

Purpose:

Configure authentication and authorization.

Must include:

Basic Authentication
Role-based endpoint access
In-memory users
BCrypt password encoding
Stateless session policy
CSRF disabled
Swagger endpoints allowed publicly

20.3 OpenApiConfig

Purpose:

Configure Swagger/OpenAPI documentation.

Must include:

```
API title
API version
API description
Basic Auth security scheme
```

21. Application Configuration Requirements

The `application.properties` file must include:

Property	Purpose
<code>spring.datasource.url</code>	Database connection URL
<code>spring.datasource.username</code>	Database username
<code>spring.datasource.password</code>	Database password
<code>spring.jpa.hibernate.ddl-auto</code>	Table creation/update strategy
<code>spring.jpa.show-sql</code>	Show SQL in console
<code>spring.jpa.properties.hibernate.format_sql</code>	Format SQL output
<code>server.port</code>	Application port
<code>springdoc.swagger-ui.path</code>	Swagger UI path

Port must be:

8080

22. Expected Request Format

22.1 Create Department Request

The create department request must contain:

```
department_name
location
employees
```

Each employee object must contain:

```
employee_name  
email  
salary
```

22.2 Update Department Request

The update department request must contain:

```
department_name  
location  
employees
```

Each employee object must contain:

```
employee_name  
email  
salary
```

23. Expected Response Format

23.1 Department Response

The department response must contain:

```
id  
department_name  
location  
employees
```

Each employee response must contain:

```
id  
employee_name  
email  
salary
```

23.2 Pagination Response

The pagination response must contain:

```
content  
pageNumber
```

```
pageSize
totalElements
totalPages
lastPage
```

24. Testing Requirements

The project must be tested using:

```
Swagger UI
Postman
```

24.1 Required Test Cases

Mentees must test the following:

Test Case	Expected Result
Create department as ADMIN	Success
Create department as USER	Forbidden
Get all departments as USER	Success
Get all departments as ADMIN	Success
Get department by valid ID	Success
Get department by invalid ID	Not found error
Create duplicate department name	Conflict error
Create duplicate employee email	Conflict error
Update department as ADMIN	Success
Update department as USER	Forbidden
Delete department as ADMIN	Success
Delete department as USER	Forbidden
Send invalid request body	Validation error
Use invalid pagination values	Bad request error

25. Final Deliverables

Mentees must submit:

```
Complete Spring Boot project
MySQL database name
All required APIs working
```


Swagger UI working
Postman collection or screenshots
Role-based security working
Exception handling working
Validation working
Logger statements added
Pagination working
Clean package structure

26. Implementation Order

Mentees should follow this order:

1. Create Spring Boot project.
 2. Add required Maven dependencies.
 3. Configure application properties.
 4. Create MySQL database.
 5. Create entity classes.
 6. Create DTO classes.
 7. Create repository interfaces.
 8. Create exception classes.
 9. Create global exception handler.
 10. Create configuration classes.
 11. Create service interface.
 12. Create service implementation.
 13. Create controller.
 14. Add security rules.
 15. Add Swagger configuration.
 16. Add logger statements.
 17. Run the application.
 18. Test APIs in Swagger.
 19. Test APIs in Postman.
 20. Verify database records.
-

27. Completion Criteria

The project will be considered complete only when:

Department can be created with multiple employees.
Department can be fetched with employees.
Department can be updated with employees.
Department can be deleted with employees.
Pagination works correctly.
Duplicate department names are blocked.
Duplicate employee emails are blocked.
Validation errors return clean responses.
Global exception handling works.
Role-based security works.
Swagger UI works.
Logger statements are present.

Code follows the required package structure.
No entity is directly exposed from controller responses.